
INITIAL PROPOSAL: DYNAMIC HIDDEN STATE HEAP FOLDING FOR EFFICIENT ATTENTION (HFOLD)

Aditya Dewan^{* 1} Pranav Ram^{* 1} Krish Mody^{* 1}

1 INTRODUCTION

Attention (particularly self-attention) has grown to be a hallmark of modern deep learning, allowing vectors (and particularly neural network weights and token representations) to weigh themselves based on the relative relevance of a set of similarly-linearly-transformed vectors. This is particularly useful for sequence modelling and language tasks, allowing each input token to take into context other tokens to learn higher-order relationships between text and other modalities, and hence why attention has become a quintessential component of the Transformer architecture (Vaswani et al., 2017) and its corollaries (GPT, Qwen, Llama, etc). Consequently, making the underlying attention mechanism more efficient has long-reaching effects across the deployment, feasibility, and scalability of modern Large Language Models (LLMs) and transformers in general.

2 PROBLEM

Attention, while a key component of modern deep learning and transformer architectures, is hampered by its widely recognized bottleneck of all-pair dot-product computation, a quadratic time-complexity operation. Yet, attention’s usefulness has driven many to devise more efficient ways of approximating said computation, such as sliding window attention which only looks, for a fixed parameter w , at $\frac{w}{2} + 1$ previous tokens (Beltagy et al., 2020) in autoregressive text generation, transformers with a linear map approximation rather than softmax (Katharopoulos et al., 2020) across all tokens, as well as alternative State Space Model (SSM) based architectures (Gu et al., 2022). All three of these aim to sequence model in $O(n)$ (or $O(w)$ in the sliding window case) while still maintaining at least some of the benefits gleaned by standard attention mechanisms. Particularly, sliding window attention enjoys high popularity in modern industry. A

key limitation, however, is that limiting the autoregressive window degrades direct token-recall-based performance and hinders longer-term generalization as well as conditioning of text output based on user input prompts, an issue that is exacerbated as sequence and context length increase. In this work, we propose a novel sliding-window-attention algorithm maintaining a dynamic heap of recursively folding hidden states that a) preserves the efficient attention computational properties of its predecessor, making it apt for deployment in low-compute, low-resource systems, while b) allowing compressed representations high-relevance vectors that lie outside the denoted sliding window to be incorporated into the end attention mechanism, enabling better performance on context-retrieval and generalization tasks.

3 STATUS QUO

There have been several lines of prior work focused on trying to reduce the quadratic cost of full self-attention while preserving long-range reasoning, a major flaw in the common approach of Flash Attention (Dao et al., 2022). These focuses fall mainly into two groups: fixed sparse/local patterns and learned sparse routing. Longformer (Beltagy et al., 2020) falls into the first group by replacing dense attention with a sliding-window pattern plus a small number of task-specific global tokens. While it achieves linear scaling on long documents, its long-range connectivity is largely hand-designed rather than learned. BigBird (Zaheer et al., 2020) similarly uses a sparse pattern combining local, random, and global attention, but its nonlocal links are still mostly fixed or randomized. Routing Transformer (Roy et al., 2021) falls into the second group, moving closer to learned sparsity by clustering tokens and attending within content-based groups. This allows them to reduce attention cost, however, this routing must be recomputed dynamically during training and inference, adding more complexity. Attention Residuals (2026) (Kimi Team: Chen et al., 2026) is also relevant because its central idea relates closely to our project. This work replaces uniform residual accumulation with learned, input-dependent attention over earlier layer representations.

^{*}Equal contribution ¹School of Computer Science, Carnegie Mellon University, Pennsylvania, USA. Correspondence to: Aditya Dewan <adewan2@andrew.cmu.edu>, Krish Mody <kmody@andrew.cmu.edu>, Pranav Ram <pram@andrew.cmu.edu>.

4 HIGH-LEVEL PLAN

We propose the following high-level algorithm. Let T be a max-heap. At every timestep, during sliding window attention, we look at a sequence of k tokens. For a decoder-only transformer, this would be the previous k tokens, while for an encoder-only transformer the current token occurs at the midpoint of the k tokens, looking symmetrically forwards and backwards throughout the sequence as is available. We enforce that the size of the heap must be a constant s .

At every iteration, we add the key vectors corresponding to the highest q attention scores after computing the sliding window attention operation to T (where the key of the heap is the attention score, and the value is the corresponding key vector that led to said node). At each iteration, we pop the top e values from said heap (where e, k, q are hyperparameters).

We then attend to these keys as well in the current sliding window attention computation, effectively allowing the current window to attend tokens that previously sparked high-activation states. Observe that the same key vector can be added multiple times given the improbability of two key vectors having the same attention score, meaning that if the same underlying key vector is consistently deemed as important, it will be considered by more future sliding window token computations). Of course, we will add a separate linear transformation to obtain the appropriate value and attention key from the given historical key vector. Subsequently, the highest q attention-score-driving key vectors from this given computation will therefore be added to the heap, pushing out q values. This means that we have pushed out $e + q$ values.

Subsequently, we do the following: to every node on the heap of size $|T|$, we add a linear combination of all of the vectors that have been removed (the original sliding window tokens removed as well as the q that were removed when we pushed our top q attention score keys) along with a gated nonlinearity, in effect creating a heap where each key is now a weighted amalgamation of the previously removed tokens. Therefore, when that key is popped in the future and attended to, said key is also conditioned on previous tokens - in effect, the key becomes a hidden state (much like an RNN hidden state) that when attended conditions on all previous folded states. This means that the sliding window algorithm can still maintain attention for $e + k$ tokens while effectively conditioning on a much larger window, which theoretically should increase long-context performance and enhance recall accuracy. This means that every attention computation views a constant number of tokens ($e + k$) for all $n - k + 1$ sliding windows, making the final algorithm cost $O(n)$ as desired (since popping and pushing k and q nodes to T is $(e + q) \log s \in O(\log s) \in O(1)$ since s is a fixed constant) while still incorporating information beyond

the sliding window in a methodological manner as desired.

We will benchmark this new algorithm (HFold, Dynamic Hidden State Heap Folding for Efficient Attention) on fine-tuning tasks along with regular sliding-window attention, and regular self attention on the Project Gutenberg 19 (PG-19) dataset, historically leveraged for long-context task benchmarking. We will augment Pythia-160M (Biderman et al., 2023) with each of the aforementioned attention mechanisms and fine-tune on PG-19 while measuring perplexity at different token counts and total FLOPs. We will do the same with GPT-2 and compare with LongFormer (Beltagy et al., 2020) (time permitting) on the SCROLLS dataset (Shaham et al., 2022) as well, reporting the same metric.

REFERENCES

- Beltagy et al. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- Biderman et al. Pythia: A suite for analyzing large language models across training and scaling. In *Proceedings of the 40th International Conference on Machine Learning*, pp. 2397–2430. PMLR, 2023.
- Dao et al. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pp. 16344–16359, 2022.
- Gu, A., Goel, K., and Ré, C. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022.
- Katharopoulos et al. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, pp. 5156–5165. PMLR, 2020.
- Kimi Team: Chen, G., Zhang, Y., Su, J., and Xu, e. a. Attention residuals. *arXiv preprint arXiv:2603.15031*, 2026.
- Roy et al. Efficient content-based sparse attention with routing transformers. In *Transactions of the Association for Computational Linguistics*, volume 9, pp. 53–68, 2021.
- Shaham et al. SCROLLS: Standardized Comparison over long language sequences. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 12007–12021, 2022.
- Vaswani et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Zaheer et al. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, volume 33, pp. 17283–17297, 2020.